

CI/CD PIPELINE DEPLOYMENT WITH DOCKER & JENKINS ON AWS

Automated Containerization, Versioned Registry Delivery & Zero-Downtime Deployment Report

Project Title: Automated CI/CD Deployment to Docker via Jenkins	Target Platform: AWS EC2 (Ubuntu 22.04 LTS)
DevOps Engineer: Rokit S (rokit-cloud)	Application: Kanban Style Task Manager (Port 3000)

1. Executive Summary & CI/CD Architecture Blueprint

This technical report documents the end-to-end design, implementation, and automated deployment of a modern containerized web application using Jenkins and Docker on Amazon Web Services (AWS). Commits to GitHub trigger Jenkins via Webhooks to build, security-test, push to Docker Hub, and deploy with zero downtime.

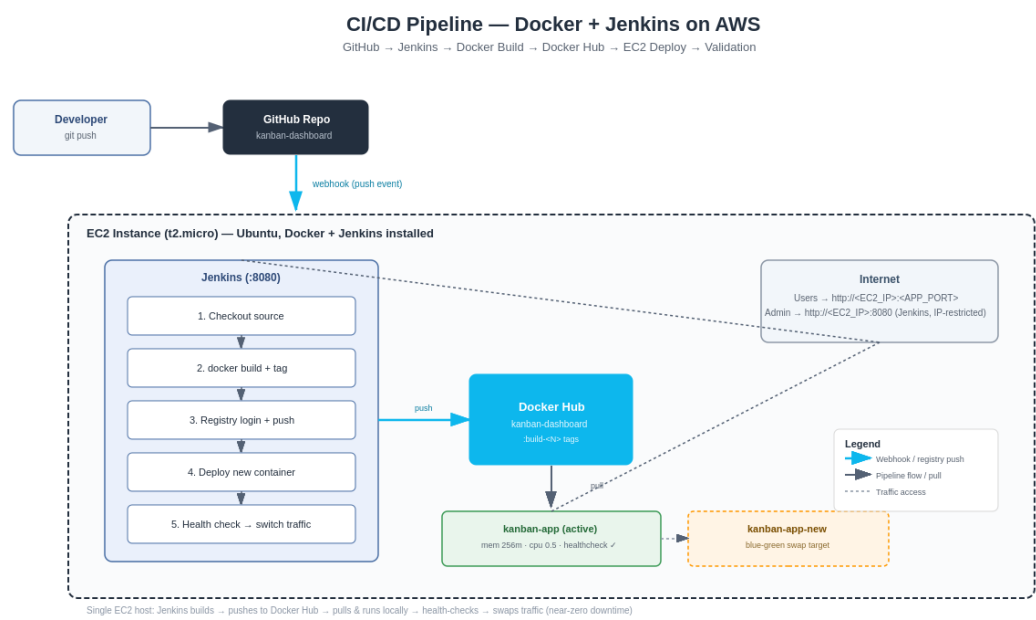


Figure: CI/CD Pipeline Architecture Diagram - Docker + Jenkins on AWS

2. Stage 1: Application Setup & Local Verification

The application repository was cloned and tested locally. Node.js dependencies were installed, and the local Vite development server started successfully on port 5173.

```
git clone https://github.com/rokit-cloud/kanban-dashboard.git
cd kanban-dashboard
npm install
npm run dev
# Local: http://localhost:5173/
```

```
C:\WINDOWS\system32\cmd. x + v

56 packages are looking for funding
  run 'npm fund' for details

found 0 vulnerabilities

C:\Users\Admin\kanban-dashboard>npm run
Scripts available in kanban-style-task-manager@0.0.0 via 'npm run':
  dev
  vite
  build
    tsc -b && vite build
  lint
    eslint .
  preview
    vite preview

C:\Users\Admin\kanban-dashboard>npm run dev

> kanban-style-task-manager@0.0.0 dev
> vite

11:00:16 am [vite] (client) [optimizer] scanning dependencies...

VITE v8.3.0 ready in 3921 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```

Figure: Vite Development Server Active on Localhost Port 5173

2.1 Local Web Application Output

The Kanban Dashboard web interface loads correctly on localhost port 5173, confirming application functional integrity.

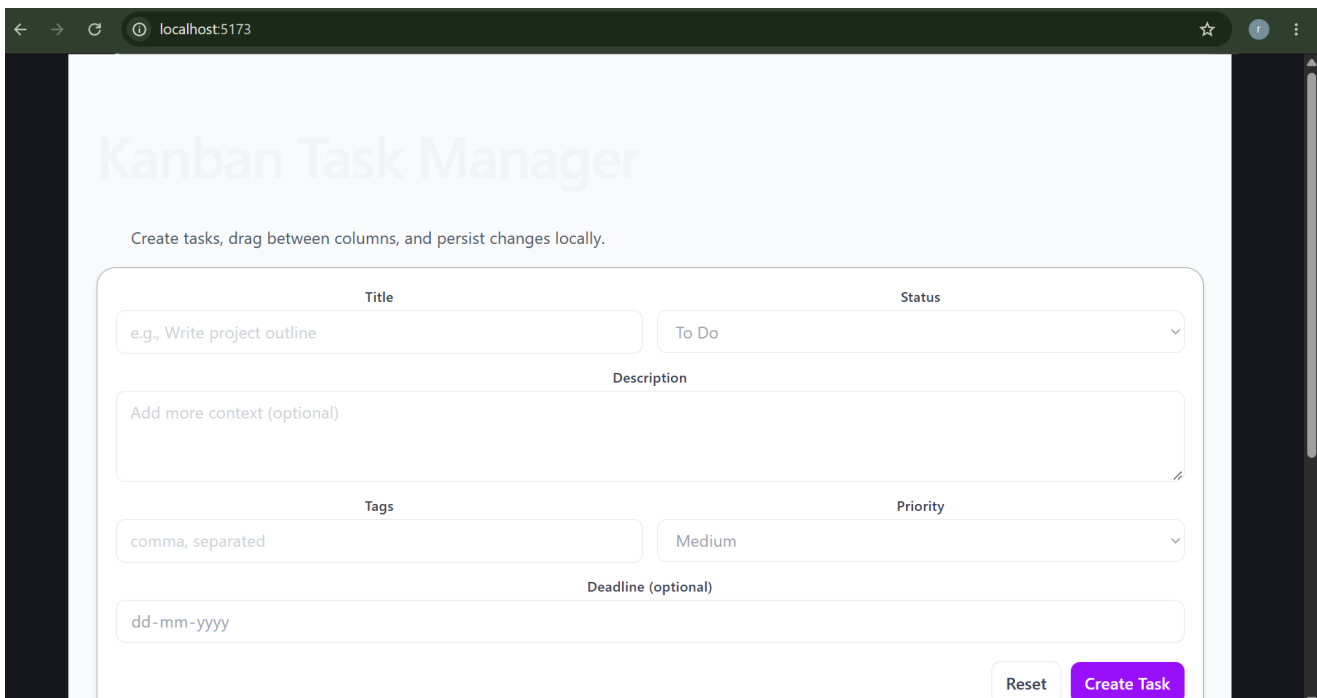


Figure: Localhost Browser Rendering of Kanban Style Task Manager

3. Stage 2: Docker Containerization (Multi-Stage Build)

A multi-stage Dockerfile was created with build and runtime stages. The application was built into a local Docker image and tested on container port 3000.

```
docker build -t kanban-app:local .
docker run -d -p 3000:3000 --name kanban-test kanban-app:local
curl http://localhost:3000
```

```
PS C:\Users\Admin\kanban-dashboard> docker build -t kanban-app:local .
[+] Building 5.4s (13/13) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile                0.1s
=> => transferring dockerfile: 393B                               0.0s
=> [internal] load metadata for docker.io/library/node:22-alpine  3.0s
=> [auth] library/node:pull token for registry-1.docker.io        0.0s
=> [internal] load .dockerignore                                  0.0s
=> => transferring context: 2B                                     0.0s
=> [build 1/6] FROM docker.io/library/node:22-alpine@sha256:b6f26b36c8ff49624cfdac716b8ea1138d606df02586a77d364b  0.1s
=> => resolve docker.io/library/node:22-alpine@sha256:b6f26b36c8ff49624cfdac716b8ea1138d606df02586a77d364bb5536a  0.1s
=> [internal] load build context                                  1.3s
=> => transferring context: 603.22kB                               1.2s
=> CACHED [build 2/6] WORKDIR /app                                0.0s
=> CACHED [build 3/6] COPY package*.json ./                       0.0s
=> CACHED [build 4/6] RUN npm install                             0.0s
=> CACHED [build 5/6] COPY . .                                    0.0s
=> CACHED [build 6/6] RUN npm run build                           0.0s
=> CACHED [runtime 3/3] COPY --from=build /app ./                 0.0s
=> exporting to image                                             0.3s
=> => exporting layers                                             0.0s
=> => exporting manifest sha256:4c6186321f6d48ae7e88cc0a8ab98bca0c6cf2265ceaf7fec4b654341cb24949  0.0s
=> => exporting config sha256:4ed39a604c1c1493619ff99b63ae30e9265dbfdea868d1f286ec52ef30962ea7  0.0s
=> => exporting attestation manifest sha256:12ffcee9753466682d26c160e0a02bb1d1e3b7bf95298a49126578329daba7d3  0.1s
=> => exporting manifest list sha256:1a3db9dc8772a2e9c0f5bf2a674785d1c38f23687f71962246371dacd55635db  0.1s
=> => naming to docker.io/library/kanban-app:local               0.0s
=> => unpacking to docker.io/library/kanban-app:local            0.0s
PS C:\Users\Admin\kanban-dashboard>
```

Figure: Multi-Stage Docker Build Finished Successfully

3.1 Local Container HTTP Response

Local curl command verifies container is running and serving HTTP 200 OK responses.

```
PS C:\Users\Admin\kanban-dashboard> curl http://localhost:3000

Security Warning: Script Execution Risk
Invoke-WebRequest parses the content of the web page. Script code in the web page might be run when the page is
parsed.
RECOMMENDED ACTION:
Use the -UseBasicParsing switch to avoid script code execution.

Do you want to continue?

[Y] Yes [A] Yes to All [N] No [L] No to All [S] Suspend [?] Help (default is "N"): y

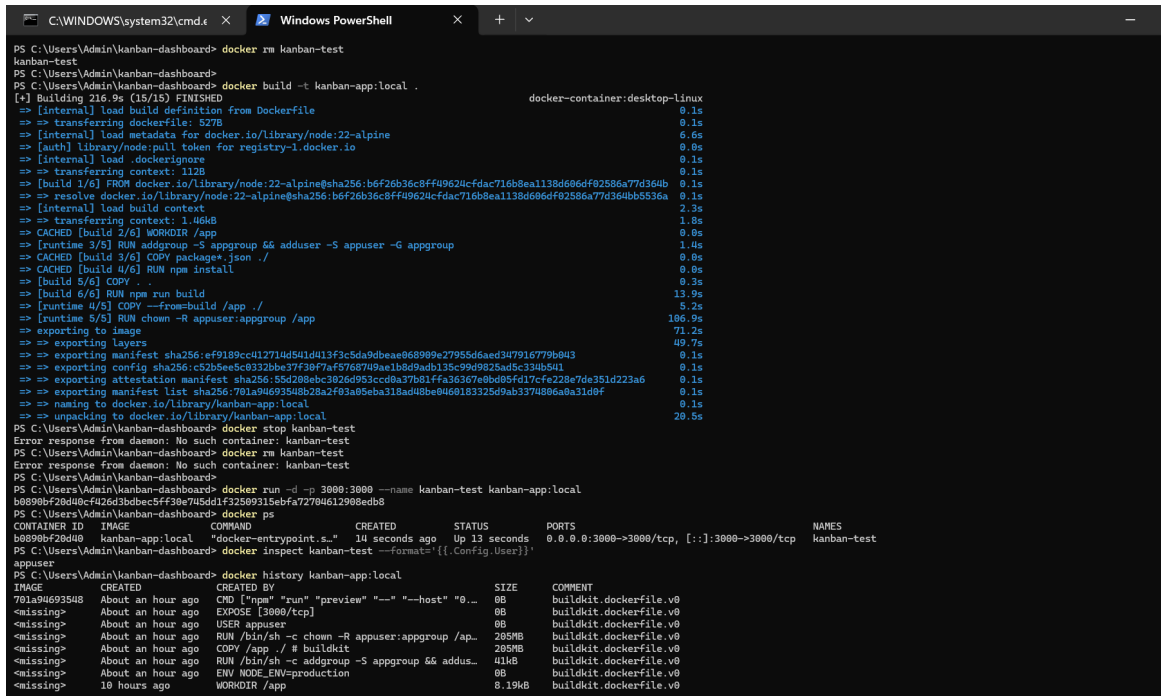
StatusCode      : 200
StatusDescription : OK
Content          : <!doctype html>
                  <html lang="en">
                    <head>
                      <meta charset="UTF-8" />
                      <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
                      <meta name="viewport" content="width=device-width, init...
RawContent      : HTTP/1.1 200 OK
                  Vary: Origin
                  Connection: keep-alive
                  Keep-Alive: timeout=5
                  Content-Length: 488
                  Cache-Control: no-cache
                  Content-Type: text/html
                  Date: Tue, 22 Sep 2026 06:55:05 GMT
                  ETag: W/"1e8-y...
```

Figure: Running Container Responding with HTTP 200 OK via curl

4. Stage 3: Container Security Hardening (Non-Root User)

To prevent root privilege escalation, a dedicated non-root user (appuser:appgroup) was added to the runtime container stage. Docker inspect confirmed execution under appuser.

```
docker inspect kanban-test --format='{{.Config.User}}'
# Output: appuser
```



```
PS C:\Users\Admin\kanban-dashboard> docker rm kanban-test
kanban-test
PS C:\Users\Admin\kanban-dashboard>
PS C:\Users\Admin\kanban-dashboard> docker build -t kanban-app:local .
[*] Building 216.9s (15/15) FINISHED                                docker-container:desktop-linux
=> [internal] load build definition from Dockerfile                  0.1s
=> => transferring dockerfile: 527B                                  0.1s
=> [internal] load metadata for docker.io/library/node:22-alpine    0.6s
=> [auth] library/node:pull token for registry-1.docker.io          0.8s
=> [internal] load .dockerignore                                     0.1s
=> => transferring context: 112B                                       0.1s
=> [build 1/6] FROM docker.io/library/node:22-alpine@sha256:b6f26b36c8ff49624cfdac716b8e1138d686df02586a77d364b 0.1s
=> => resolve docker.io/library/node:22-alpine@sha256:b6f26b36c8ff49624cfdac716b8e1138d686df02586a77d364b5536a 0.1s
=> [internal] load build context                                     2.3s
=> => transferring context: 1.46kB                                       1.8s
=> CACHED [build 2/6] WORKDIR /app                                   0.0s
=> [runtime 3/5] RUN addgroup -S appgroup && adduser -S appuser -G appgroup 1.4s
=> CACHED [build 3/6] COPY package*.json ./                          0.0s
=> CACHED [build 4/6] RUN npm install                                0.0s
=> [build 5/6] COPY . .                                              0.3s
=> [build 6/6] RUN npm run build                                     13.9s
=> [runtime 4/5] COPY --from=build /app . /                          5.2s
=> [runtime 5/5] RUN chown -R appuser:appgroup /app                 186.9s
=> exporting to image                                                71.2s
=> => exporting layers                                                  49.7s
=> => exporting manifest sha256:ef9189cc41271d4541d413f3c5da9dbee968999e77955d6aed347916779b043 0.1s
=> => exporting config sha256:c52b5ee5c832bbe37f387af5f68749aeb8d9adb135c99d9825ad5c334b541 0.1s
=> => exporting attestation manifest sha256:55d288ebc3026d983cc0a37b81ffa36367e0bd085fd17cfe228a7de351d223a6 0.1s
=> => exporting manifest list sha256:701a94693540b2ba2f03a05eba31bad48be0460183325c9ab337486a0a31d0f 0.1s
=> => naming to docker.io/library/kanban-app:local                   0.1s
=> => unpacking to docker.io/library/kanban-app:local                 20.5s
PS C:\Users\Admin\kanban-dashboard> docker stop kanban-test
Error response from daemon: No such container: kanban-test
PS C:\Users\Admin\kanban-dashboard> docker rm kanban-test
Error response from daemon: No such container: kanban-test
PS C:\Users\Admin\kanban-dashboard>
PS C:\Users\Admin\kanban-dashboard> docker run -d -p 3000:3000 --name kanban-test kanban-app:local
b0890bf28d08-f426c3db0bc8ff30e745dd1f32589315ebfa7270461298edeb8
PS C:\Users\Admin\kanban-dashboard> docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED       STATUS       PORTS                    NAMES
b0890bf28d08   kanban-app:local "docker-entrypoint.s..." 14 seconds ago Up 13 seconds 0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp kanban-test
PS C:\Users\Admin\kanban-dashboard> docker inspect kanban-test --format='{{.Config.User}}'
appuser
PS C:\Users\Admin\kanban-dashboard> docker history kanban-app:local
IMAGE          CREATED          CREATED BY          SIZE      COMMENT
701a94693540   About an hour ago CMD ["npm" "run" "preview" "--" "--host" "0.0.0.0"] 0B        buildkit.dockerfile.v0
<missing>      About an hour ago EXPOSE [3000/tcp] 0B        buildkit.dockerfile.v0
<missing>      About an hour ago USER appuser      0B        buildkit.dockerfile.v0
<missing>      About an hour ago RUN /bin/sh -c chown -R appuser:appgroup /app... 285MB     buildkit.dockerfile.v0
<missing>      About an hour ago COPY /app . / # buildkit 285MB     buildkit.dockerfile.v0
<missing>      About an hour ago RUN /bin/sh -c addgroup -S appgroup && addus... 41kB     buildkit.dockerfile.v0
<missing>      About an hour ago ENV NODE_ENV=production 0B        buildkit.dockerfile.v0
<missing>      10 hours ago     WORKDIR /app       8.19kB    buildkit.dockerfile.v0
```

Figure: Security Verification: Non-Root 'appuser' Confirmed via docker inspect

5. Stage 4: Docker Hub Registry Integration & Semantic Versioning

A public repository was established on Docker Hub (rokit45/kanban-dashboard). The local image was version-tagged as build-1, pushed, and tested with a fresh pull.

```
docker login -u rokit45
docker tag kanban-app:local rokit45/kanban-dashboard:build-1
docker push rokit45/kanban-dashboard:build-1
docker rmi rokit45/kanban-dashboard:build-1
docker pull rokit45/kanban-dashboard:build-1
```

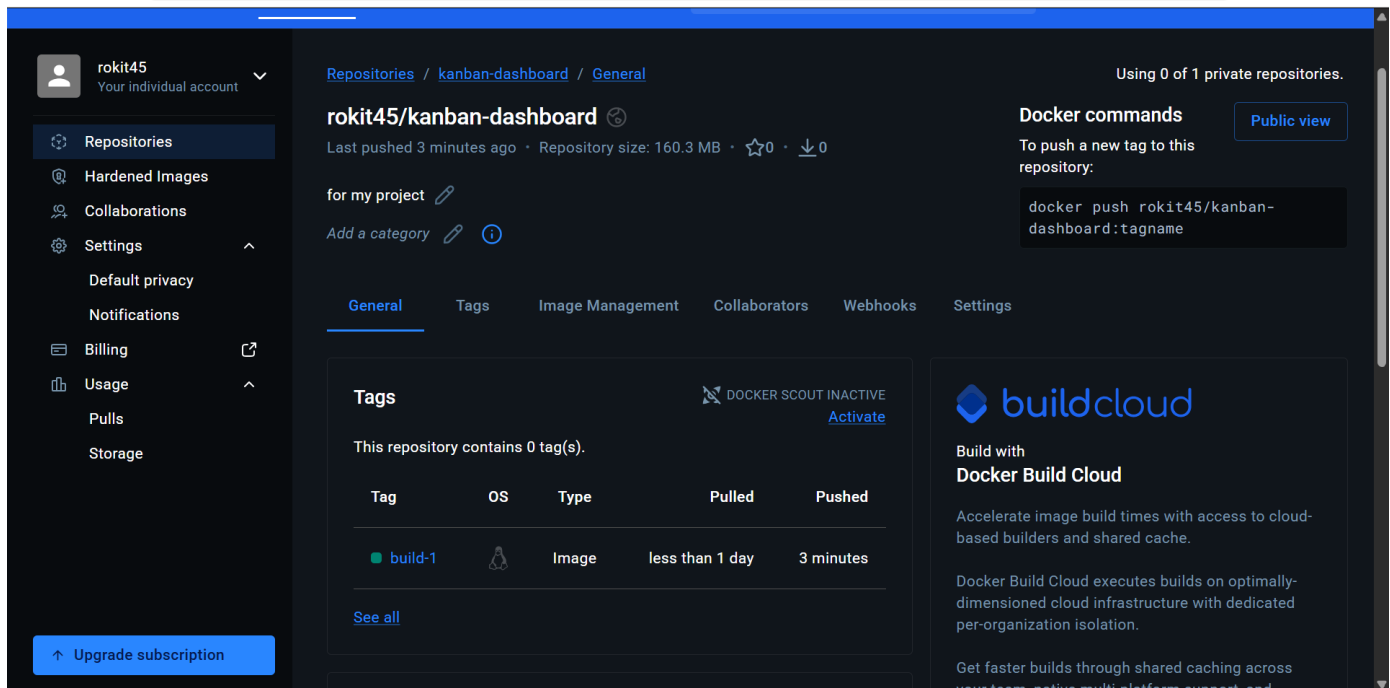


Figure: Docker Hub Repository with Tag 'build-1' Published

5.1 Registry Push & Pull Validation

Terminal output confirms successful authentication, layer uploading, and fresh pull from Docker Hub.

```
PS C:\Users\Admin\kanban-dashboard>
PS C:\Users\Admin\kanban-dashboard> docker tag kanban-app:local rokit45/kanban-dashboard:build-1
PS C:\Users\Admin\kanban-dashboard> docker push rokit45/kanban-dashboard:build-1
The push refers to repository [docker.io/rokit45/kanban-dashboard]
55d3bbcf2e97: Pushed
44136fa355b3: Pushed
e2de96513ba9: Pushed
1a3a899a8294: Pushed
ec179e2ee072: Pushed
65213f1a3de8: Pushed
cfb01f0de303: Pushed
243db053c65a: Pushed
f8e85b5cdf9d: Pushed
6667253abd83: Pushed
build-1: digest: sha256:701a94693548b28a2f03a05eba318ad48be0460183325d9ab3374806a0a31d0f size: 856
PS C:\Users\Admin\kanban-dashboard> docker rmi rokit45/kanban-dashboard:build-1
Untagged: rokit45/kanban-dashboard:build-1
PS C:\Users\Admin\kanban-dashboard> docker tag kanban-app:local rokit45/kanban-dashboard:build-1
PS C:\Users\Admin\kanban-dashboard> docker rmi rokit45/kanban-dashboard:build-1
Untagged: rokit45/kanban-dashboard:build-1
PS C:\Users\Admin\kanban-dashboard> docker pull rokit45/kanban-dashboard:build-1
build-1: Pulling from rokit45/kanban-dashboard
Digest: sha256:701a94693548b28a2f03a05eba318ad48be0460183325d9ab3374806a0a31d0f
Status: Downloaded newer image for rokit45/kanban-dashboard:build-1
docker.io/rokit45/kanban-dashboard:build-1
PS C:\Users\Admin\kanban-dashboard>
```

Figure: Terminal Push, Removal, and Fresh Image Pull from Docker Hub

6. Stage 5: AWS EC2 Infrastructure Setup (Docker + Jenkins Host)

An AWS EC2 instance (Ubuntu 22.04 LTS, Public IP 13.201.81.29) was launched in ap-south-1b. Docker and Jenkins were installed, and Jenkins was granted Docker daemon privileges.

```
sudo apt update -y && sudo apt install -y docker.io openjdk-17-jre jenkins
sudo usermod -aG docker jenkins
sudo systemctl restart jenkins
```

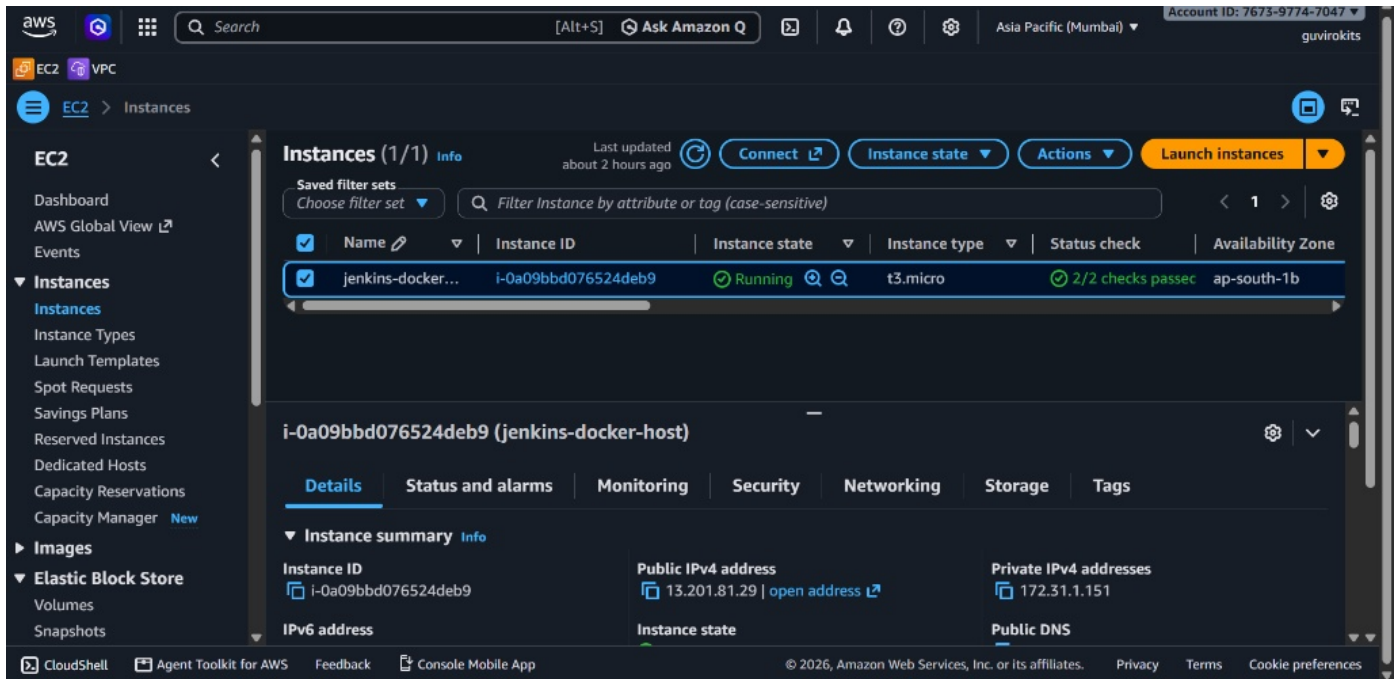


Figure: AWS EC2 Instance 'jenkins-docker-server' Running (2/2 checks passed)

6.1 Jenkins Portal Initialization

Jenkins web portal was unlocked and initialized on port 8080.

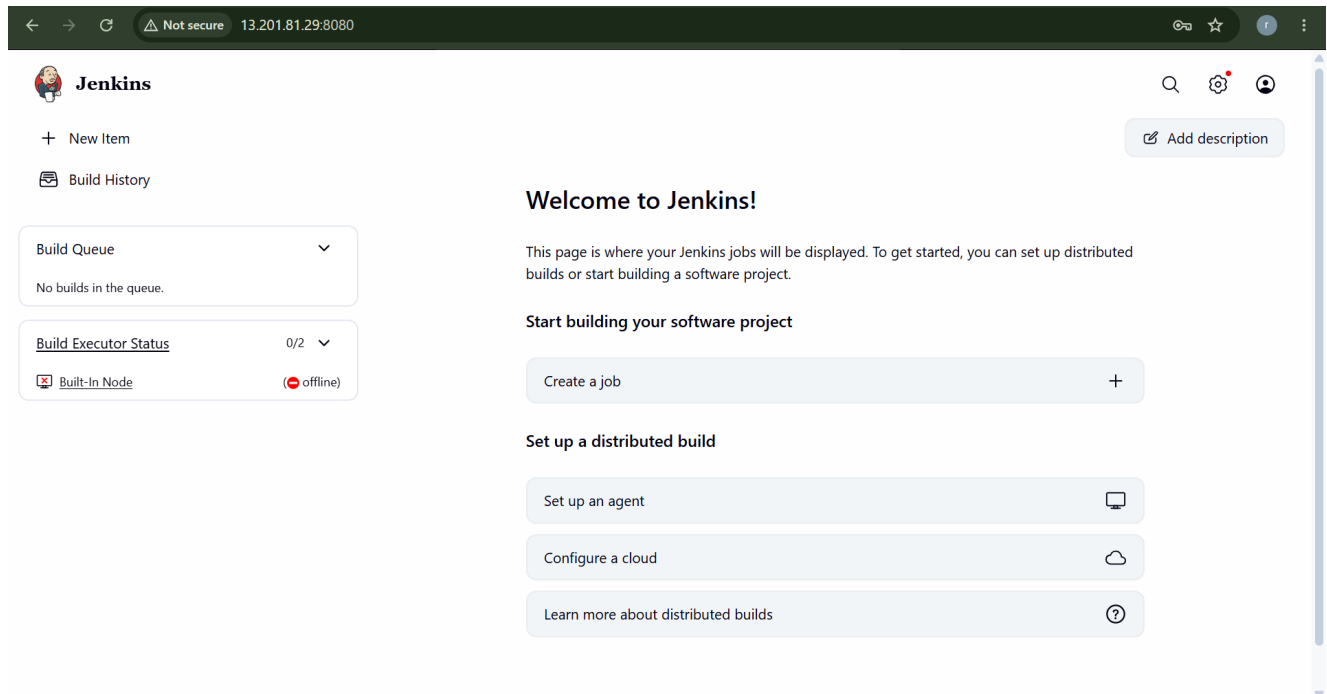


Figure: Jenkins Web Management Portal Initialized at Port 8080

7. Stages 6 & 7: Jenkins Pipeline Configuration & CI/CD Execution

Jenkins was connected to GitHub using SCM and Docker Hub credentials. A declarative pipeline executed Checkout, Build, Docker Hub Push, and EC2 Deployment.

```

pipeline {
  agent any
  environment { IMAGE_NAME = 'rokit45/kanban-dashboard'; APP_PORT = '3000' }
  stages {
    stage('Checkout') { steps { checkout scm } }
    stage('Build Docker Image') { steps { sh 'docker build -t ${IMAGE_NAME}:${BUILD_TAG} .' } }
    stage('Docker Hub Login & Push') { steps { ... } }
    stage('Deploy to EC2') { steps { ... } }
  }
}

```

The screenshot displays the Jenkins web interface for a build named 'kanban-dashboard' with ID '#3'. The interface includes a left-hand navigation menu with options like 'Status', 'Changes', 'Console Output', 'Delete build', 'Git Build Data', 'Pipeline Overview', 'Edit build information', 'Restart from Stage', 'Replay', 'Pipeline Steps', 'Workspaces', and 'Previous Build'. The main content area shows a green checkmark icon and the text '#3 (Sep 24, 2026, 9:09:19 AM)'. Below this, it indicates the build was 'Started by user rokit' and 'Replayed #2 (diff)'. The 'Git Build Data' section shows the 'Revision' as '1eb975b84390384bde3692e0ce27640b76c76c8e' and the 'Repository' as 'https://github.com/rokit-cloud/kanban-dashboard.git'. A 'No changes' message is displayed under the 'git' icon. On the right side, there are buttons for 'Add description' and 'Keep this build forever', along with a timer showing 'Started 3 min 54 sec ago' and 'Took 16 sec'. The bottom right corner of the interface shows 'REST API' and 'Jenkins 2.568.3'.

Figure: Jenkins Build #3 Completed Successfully via SCM Replay

7.1 Pipeline Stages & Console Execution Log

Pipeline Stage View and execution logs confirm complete end-to-end success.

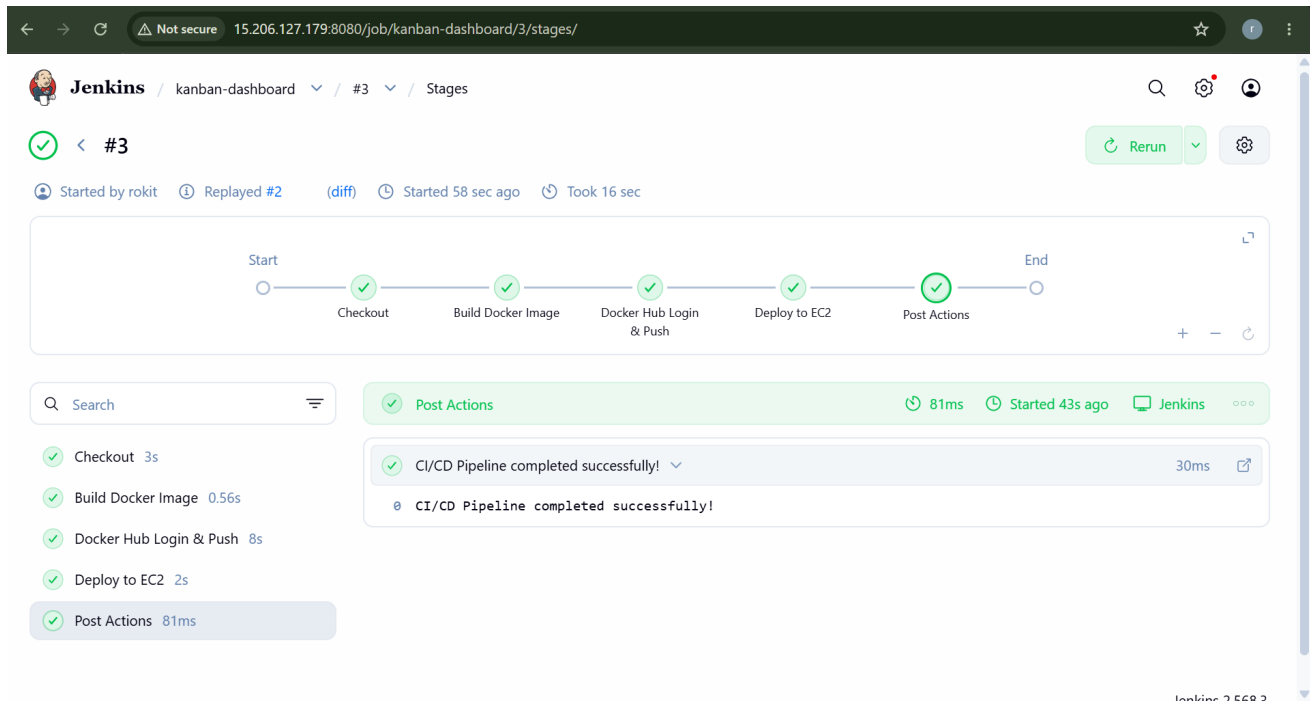


Figure: Stage View: Pipeline Stages Passed (Checkout, Build, Push, Deploy)

7.2 Pipeline Execution Output

Console logs confirm post-actions and overall SUCCESS status for Build #3.

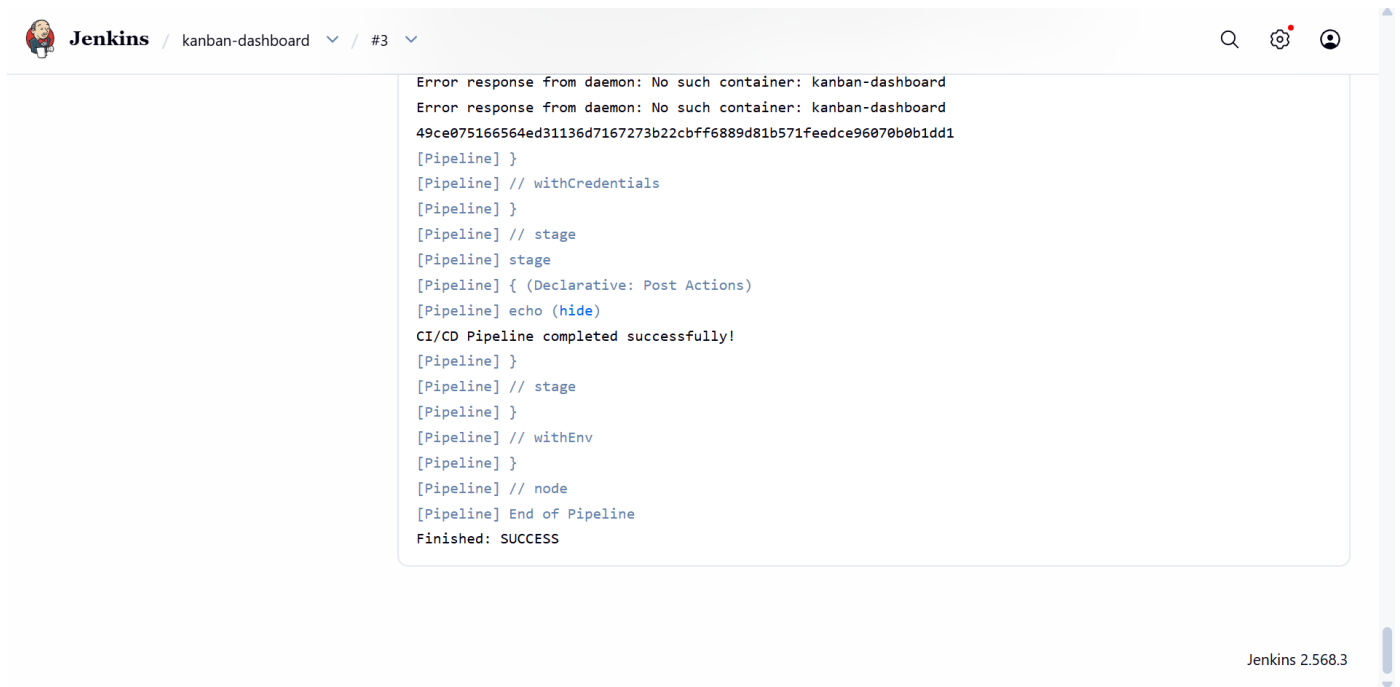


Figure: Console Output: CI/CD Pipeline Completed Successfully (Finished: SUCCESS)

8. Stage 8: Automated CI/CD Trigger via GitHub Webhooks

A GitHub Webhook was linked to the Jenkins endpoint. Pushing code to GitHub automatically triggered Jenkins Build #4 without manual intervention.

```
git add README.md
git commit -m 'test webhook trigger'
git push origin main
```

```
remote: Enumerating objects: 7873, done.
remote: Counting objects: 100% (13/13), done.
remote: Compressing objects: 100% (12/12), done.
remote: Total 7873 (delta 4), reused 3 (delta 1), pack-reused 7860 (from 1)
Receiving objects: 100% (7873/7873), 38.81 MiB | 12.11 MiB/s, done.
Resolving deltas: 100% (1655/1655), done.
PS C:\Users\Admin\Desktop> cd kanban-dashboard
PS C:\Users\Admin\Desktop\kanban-dashboard> git remote -v
origin  https://github.com/rokit-cloud/kanban-dashboard.git (fetch)
origin  https://github.com/rokit-cloud/kanban-dashboard.git (push)
PS C:\Users\Admin\Desktop\kanban-dashboard> git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Users\Admin\Desktop\kanban-dashboard> git add README.md
PS C:\Users\Admin\Desktop\kanban-dashboard> git commit -m "test webhook trigger"
[main b9d62e7] test webhook trigger
1 file changed, 2 insertions(+)
PS C:\Users\Admin\Desktop\kanban-dashboard> git push origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 325 bytes | 325.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/rokit-cloud/kanban-dashboard.git
1eb975b..b9d62e7  main -> main
PS C:\Users\Admin\Desktop\kanban-dashboard>
```

Figure: Local Git Push with Commit 'test webhook trigger'

8.1 Webhook Auto-Build Verification

Jenkins Build #4 was triggered immediately by GitHub push event.

The screenshot shows the Jenkins web interface for a build named '#4'. The top navigation bar includes the Jenkins logo, the job name 'kanban-dashboard', and the build number '#4'. On the right, there are search, settings, and user icons. Below the navigation bar, the build status is shown as a green checkmark with the text '#4 (Sep 24, 2026, 12:24:39 PM)'. To the right of the status, there are buttons for 'Add description' and 'Keep this build forever'. The left sidebar contains a list of links: 'Status', 'Changes', 'Console Output', 'Delete build #4', 'Polling Log', 'Git Build Data', 'Pipeline Overview', 'Edit build information', 'Restart from Stage', 'Replay', 'Pipeline Steps', 'Workspaces', and 'Previous Build'. The main content area shows the build details, including a 'Started by GitHub push by rokit-cloud' message, the revision 'b9d62e74f3fd67798aa0af6ecc2a2b4b61010179', and the repository 'https://github.com/rokit-cloud/kanban-dashboard.git'. It also lists the branch 'refs/remotes/origin/main' and the commit message 'test webhook trigger' by 'newrokit' at '12:24 PM 9/24/26'. On the right side of the main content area, it says 'Started 5 min 7 sec ago' and 'Took 16 sec'.

Figure: Jenkins Build #4 Automatically Triggered by GitHub Webhook Push Event

9. Stages 9 & 10: Container Telemetry, Logs & Production Health Check

Container health check was verified as healthy. Live application logs and Build #5 execution were verified.

```
docker logs kanban-dashboard --tail 50
docker inspect --format='{{.State.Health.Status}}' kanban-dashboard
# Output: healthy
```

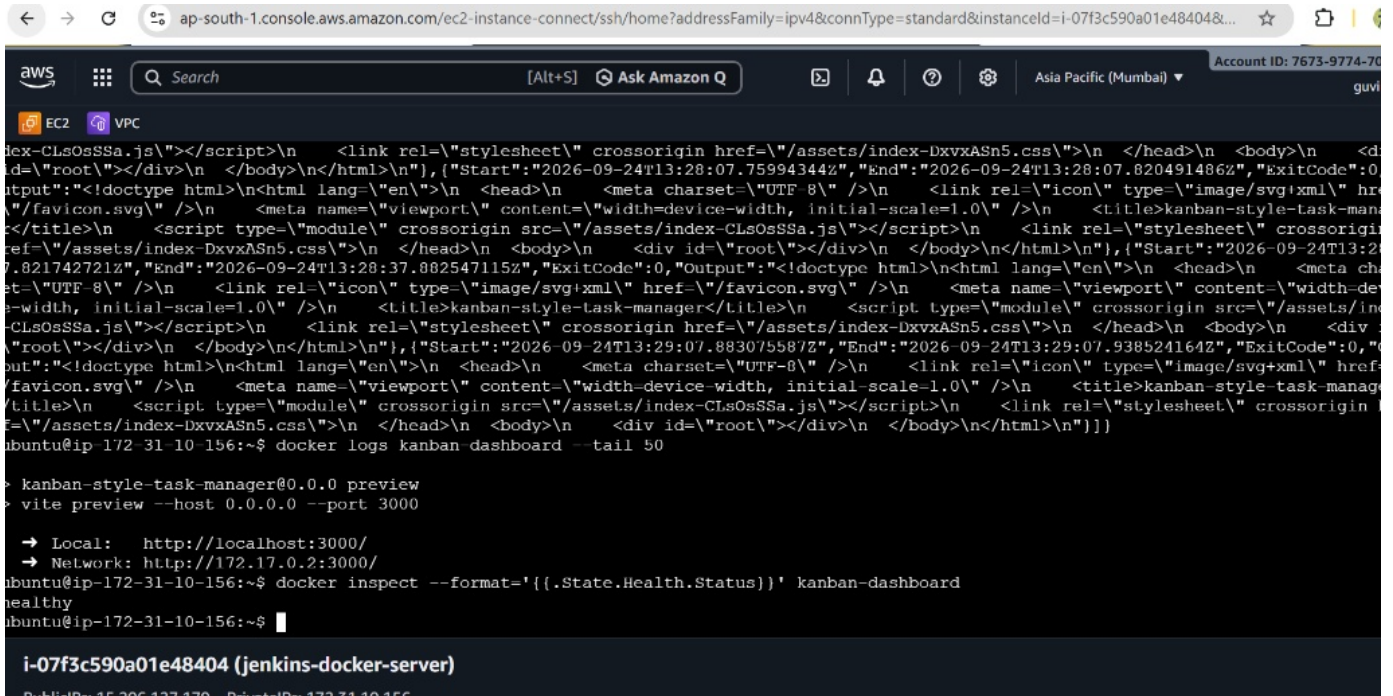


Figure: Docker Logs & Health Status Verified as 'healthy' on EC2 Host

9.1 Stage View for Build #5

Pipeline Build #5 completed in 18 seconds across all stages.

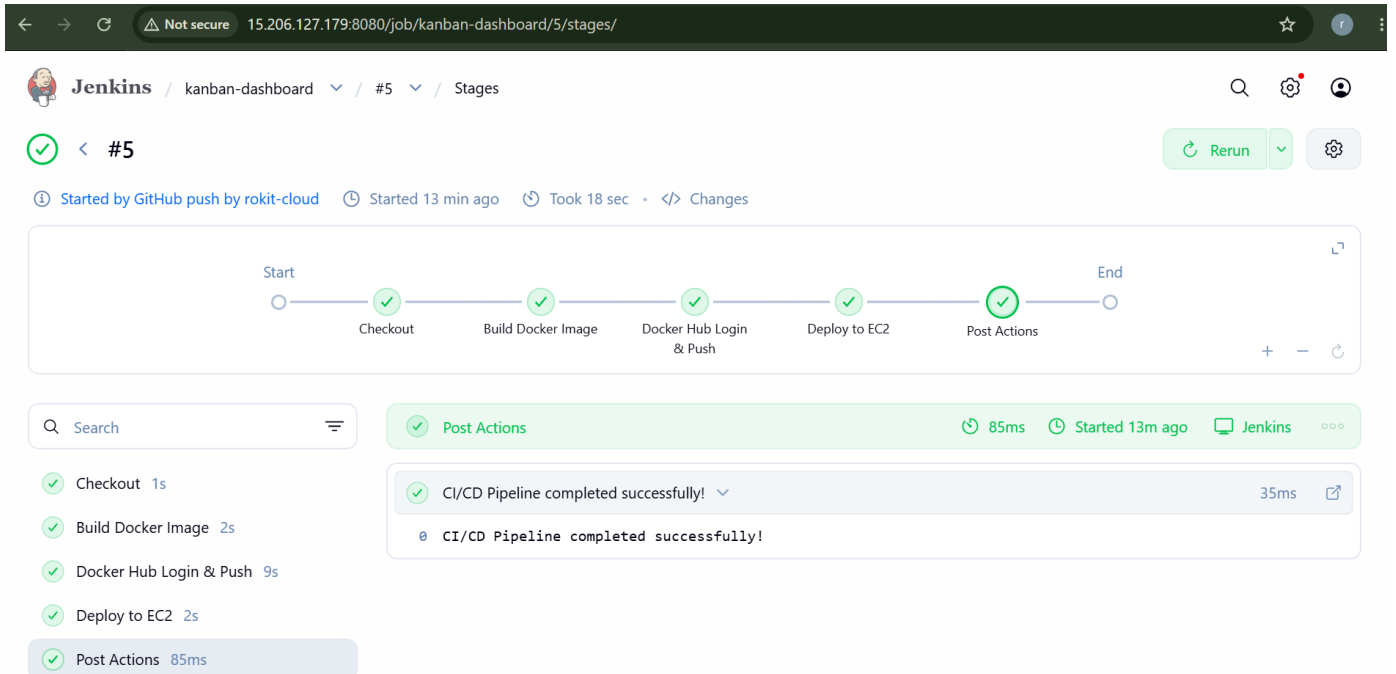


Figure: Jenkins Stage View for Build #5 Completed in 18 Seconds

10. Stage 11: Docker Resource Quotas (CPU & Memory Limits)

Strict memory (256MB) and CPU (0.5) quotas were enforced in the deployment stage. Docker inspect and docker stats verified strict resource compliance.

```
docker inspect kanban-dashboard --format='Memory: {{.HostConfig.Memory}} CPUs: {{.HostConfig.NanoCpus}}'
# Memory: 268435456 CPUs: 500000000
docker stats --no-stream kanban-dashboard
```

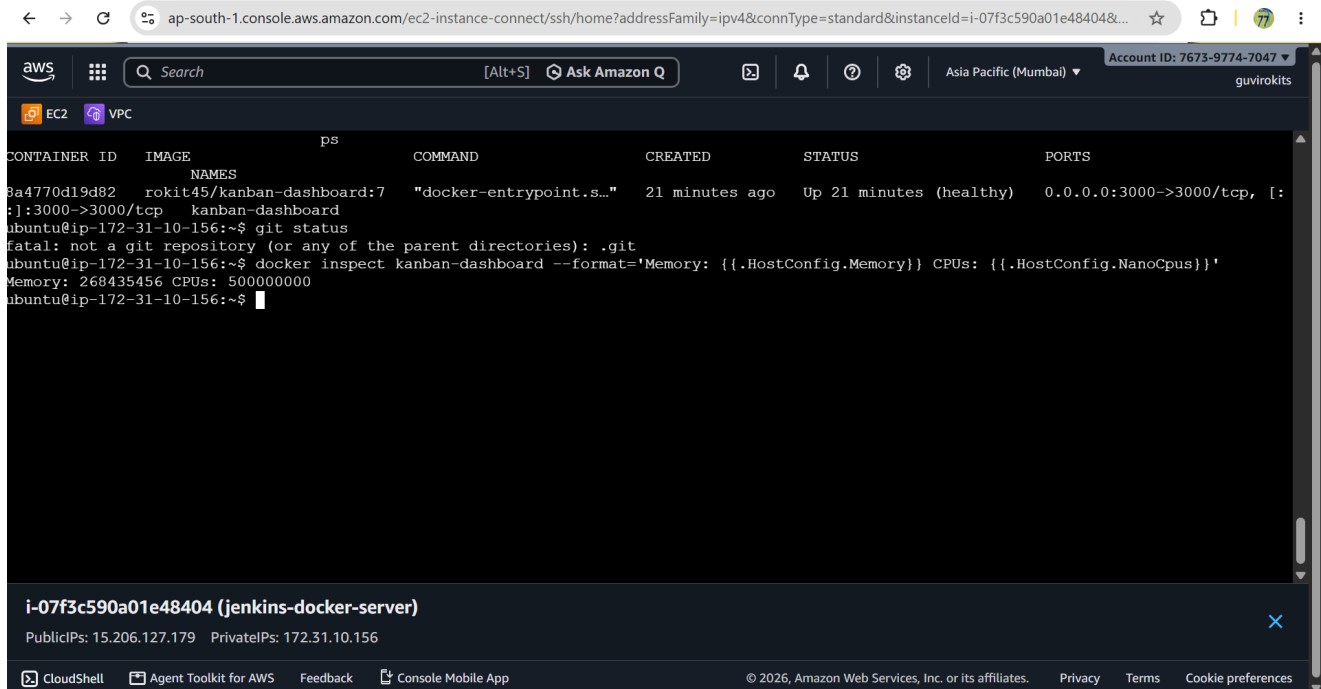


Figure: Docker Inspect Output Confirming 256MB Memory & 0.5 CPU Enforcement

10.1 Active Resource Utilization

Docker stats confirms active memory usage at 75.25MB (29.39% of 256MB limit).

